Temple Prep After School Program

By: Korey Rodi

About The Company

Temple Prep After School Program

- Temple Prep After School Program is a program for high school students to participate in college level programs
- The program is sponsored by Temple university but is not supported by, instead it operates as its own company
- Student in grades 9-12 Can enroll in any number of classes ranging from programming to the Sciences
- The courses the students take are not credit awarding meaning that the classes do not give students any sort of college credit

ER Diagram

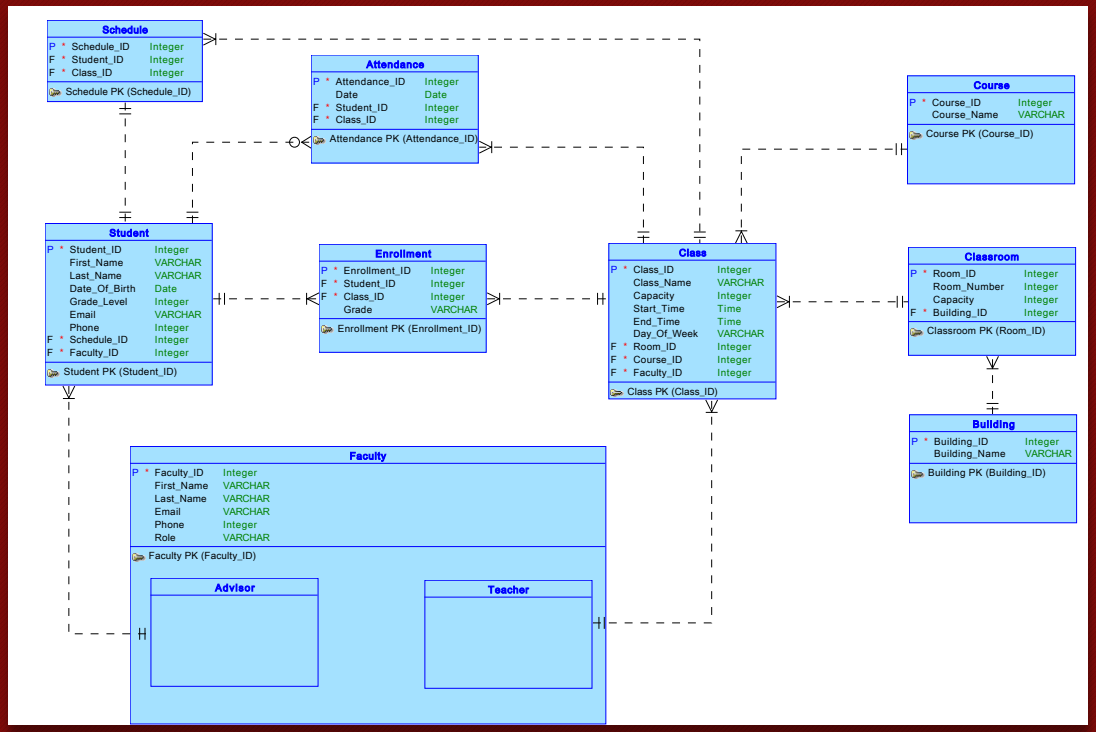
ER Diagram

Identification:

- Each student must have a unique student ID
- Each faculty Member must have a unique Faculty ID
- Each class must have a unique class ID
- Each classroom must have a unique Room ID

Student and enrollment:

- A student may enroll in multiple classes
- Each student is assigned exactly one academic advisor
- A class may have multiple students
- Attendance must be recorded for each student for the day, for each class
- Grades are calculated for each student, for each class.



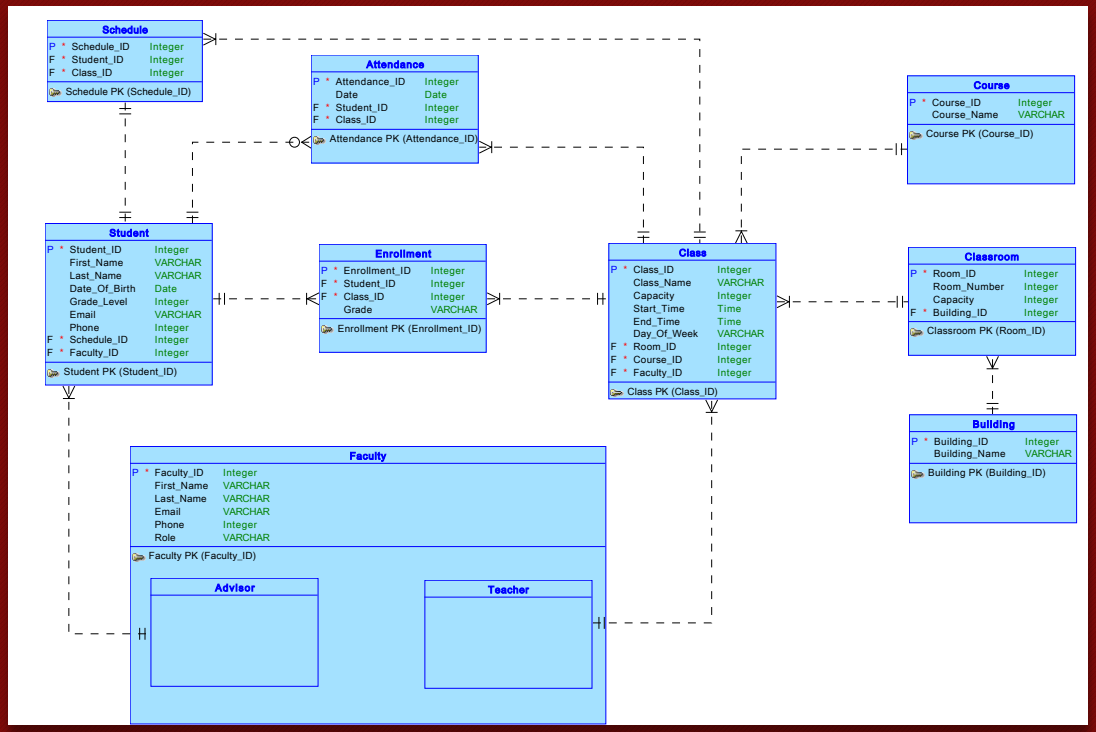
ER diagram cont.

Faculty:

- Each class is taught by exactly one Teacher
- A Teacher member may teach multiple classes
- An advisor may advise multiple students

Class and Classroom:

- Each class must be assigned to exactly one classroom
- Each classroom belongs to exactly one building
- One class per classroom at one time
- A building may contain multiple classrooms



Implementation

DDL Script

- Tables, indexes, Triggers, foreign keys, primary keys, partitions etc. are created during this step
- The ER diagram is converted to a relational Model and the DDL is extracted from there (SQL developer auto generates the script)
- The DDL script is executed to create the database objects as defined in the ER diagram (Typically errors arise and the script must be corrected)

Users > koreyrodi > Documents > GitHub > Database-Project-CIS2109 > Implementation > ProjectDDL.sql

```
1  CREATE TABLE advisor (
2      faculty_id INTEGER NOT NULL
3  );
4
5  ALTER TABLE advisor ADD CONSTRAINT advisor_pk PRIMARY KE
6
7  CREATE TABLE attendance (
8      attendance_id      INTEGER NOT NULL,
9      attendance_Date               DATE,
10     student_student_id INTEGER NOT NULL,
11     class_class_id     INTEGER NOT NULL
12 );
13
14 ALTER TABLE attendance ADD CONSTRAINT attendance_pk PRIM
15
16 CREATE TABLE building (
17     building_id   INTEGER NOT NULL,
18     building_name VARCHAR2(25) NOT NULL
19
20 );
21
22 ALTER TABLE building ADD CONSTRAINT building_pk PRIMARY
23
24 CREATE TABLE class (
25     class_id           INTEGER NOT NULL,
26     class_name         VARCHAR2(25) NOT NULL,
27     capacity           INTEGER,
28     start_time         TIMESTAMP,
```

Users > koreyrodi > Documents > GitHub > Database-Project-CIS2109 > Implementation > ProjectDDL.sql

```sql
CREATE TABLE advisor (
    faculty_id INTEGER NOT NULL
);

ALTER TABLE advisor ADD CONSTRAINT advisor_pk PRIMARY KEY ( faculty_id );

CREATE TABLE attendance (
    attendance_id      INTEGER NOT NULL,
    attendance_Date               DATE,
    student_student_id INTEGER NOT NULL,
    class_class_id     INTEGER NOT NULL
);

ALTER TABLE attendance ADD CONSTRAINT attendance_pk PRIMARY KEY ( attendance_id );

CREATE TABLE building (
    building_id   INTEGER NOT NULL,
    building_name VARCHAR2(25) NOT NULL

);

ALTER TABLE building ADD CONSTRAINT building_pk PRIMARY KEY ( building_id );
```

Users > koreyrodi > Documents > GitHub > Database-Project-CIS2109 > Implementation > ProjectDDL.sql

```sql
ALTER TABLE teacher ADD CONSTRAINT teacher_pk PRIMARY KEY ( faculty_id );

ALTER TABLE advisor
    ADD CONSTRAINT advisor_faculty_fk FOREIGN KEY ( faculty_id )
        REFERENCES faculty ( faculty_id );

ALTER TABLE attendance
    ADD CONSTRAINT attendance_class_fk FOREIGN KEY ( class_class_id )
        REFERENCES class ( class_id );

ALTER TABLE attendance
    ADD CONSTRAINT attendance_student_fk FOREIGN KEY ( student_student_id )
        REFERENCES student ( student_id );

ALTER TABLE class
    ADD CONSTRAINT class_classroom_fk FOREIGN KEY ( classroom_room_id )
        REFERENCES classroom ( room_id );

ALTER TABLE class
    ADD CONSTRAINT class_course_fk FOREIGN KEY ( course_course_id )
        REFERENCES course ( course_id );

ALTER TABLE class
    ADD CONSTRAINT class_teacher_fk FOREIGN KEY ( teacher_faculty_id )
        REFERENCES teacher ( faculty_id );

ALTER TABLE classroom
    ADD CONSTRAINT classroom_building_fk FOREIGN KEY ( building_building_id )
```

Users > koreyrodi > Documents > GitHub > Database-Project-CIS2109 > Implementation > ProjectDDL.sql

```sql
CREATE TABLE class (
    end_time           TIMESTAMP,
    day_of_week        VARCHAR2(7),
    classroom_room_id  INTEGER NOT NULL,
    course_course_id   INTEGER NOT NULL,
    teacher_faculty_id INTEGER NOT NULL
);

ALTER TABLE class ADD CONSTRAINT class_pk PRIMARY KEY ( class_id );

CREATE TABLE classroom (
    room_id              INTEGER NOT NULL,
    room_number          INTEGER,
    capacity             INTEGER,
    building_building_id INTEGER NOT NULL
);

ALTER TABLE classroom ADD CONSTRAINT classroom_pk PRIMARY KEY ( room_id );

CREATE TABLE course (
    course_id   INTEGER NOT NULL,
```

Users > koreyrodi > Documents > GitHub > Database-Project-CIS2109 > Implementation > ProjectDDL.sql

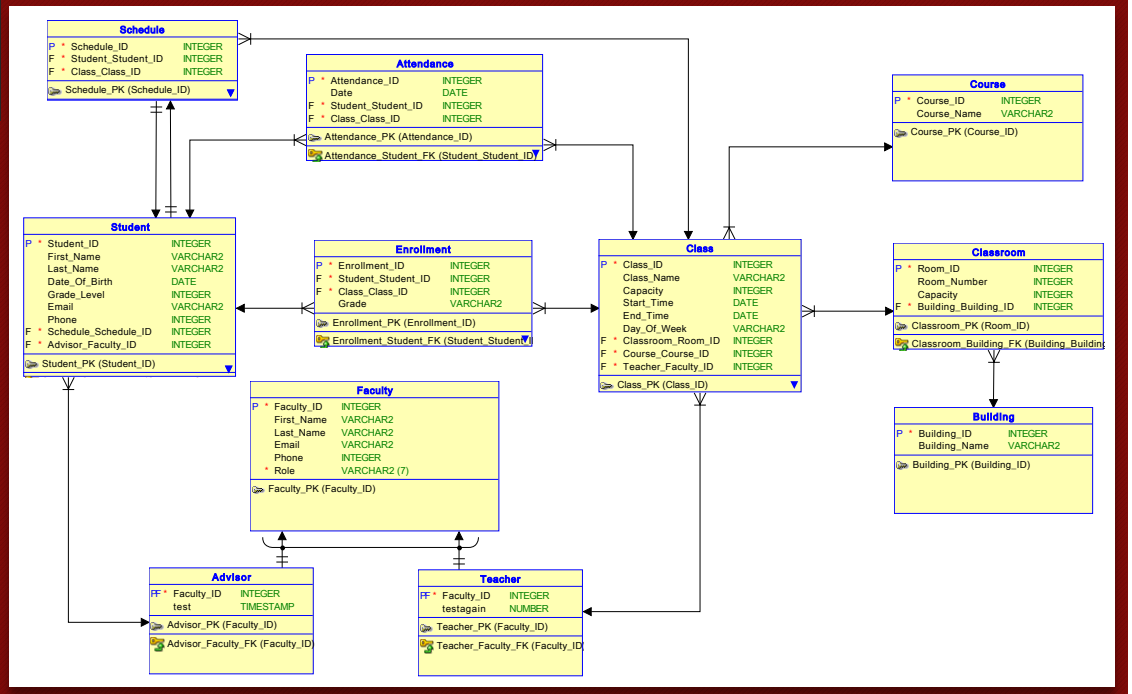
```sql
CREATE OR REPLACE TRIGGER arc_fkarc_2_advisor BEFORE
    INSERT OR UPDATE OF faculty_id ON advisor
    FOR EACH ROW
DECLARE
    d VARCHAR2(7);
BEGIN
    SELECT
        a.role
    INTO d
    FROM
        faculty a
    WHERE
        a.faculty_id = :new.faculty_id;

    IF ( d IS NULL
         OR d <> 'Advisor' ) THEN
        raise_application_error(-20223, 'FK Advisor_Faculty_FK in Table Advisor violates Arc constraint on Table
        );
    END IF;

EXCEPTION
    WHEN no_data_found THEN
        NULL;
    WHEN OTHERS THEN
        RAISE;
END;
/
```

Relational Model and Schema

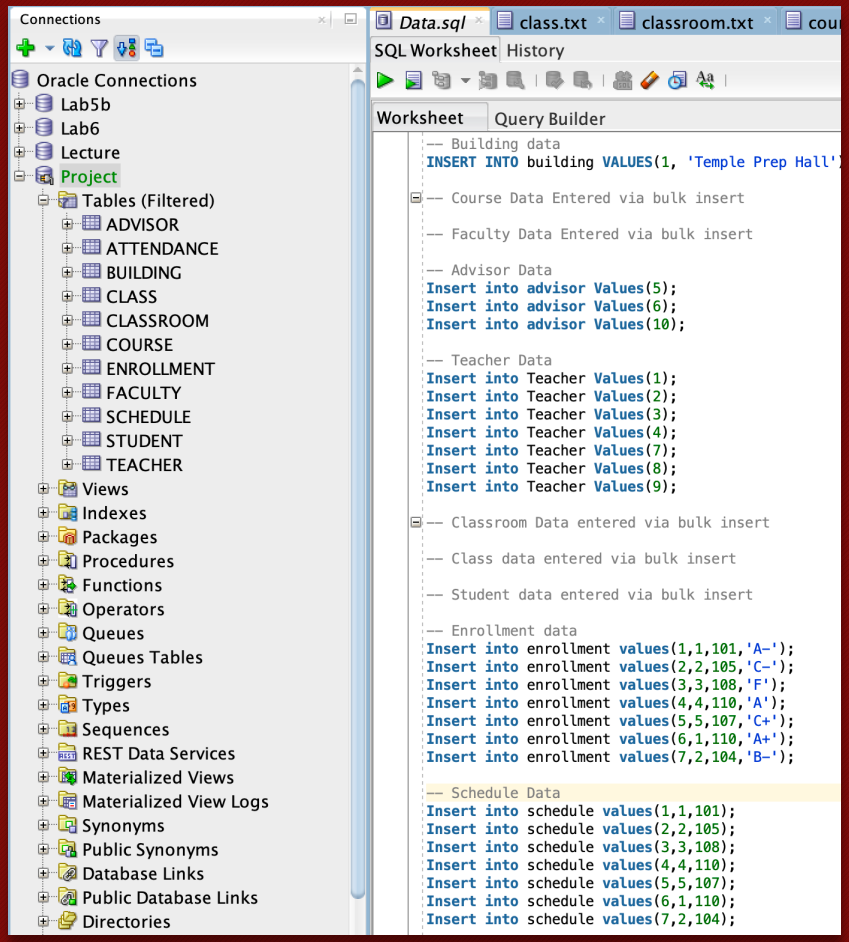
- Student (Student_ID [PK], First_Name, Last_Name, Date_Of_Birth, Grade_Level, Email, Phone, Faculty_ID [FK])
- Faculty (Faculty_ID [PK], First_Name, Last_Name, Email, Phone, Role)
- Advisor (Facutlty_ID [PK])
- Teacher (Faculty_ID [PK])
- Class (Class_ID [PK], Class_Name, Capacity, Start_Time, End_Time, Day_Of_Week, Room_ID [FK], Faculty_ID [FK], Course_ID [FK])
- Enrollment (Enrollment_ID [PK], Grade, Student_ID [FK], Class_ID [FK])
- Attendance (Attendance_ID [PK], Date, Student_ID [FK], Class_ID [FK])
- Classroom (Room_ID [PK], Room_Number, Capacity, Building_ID [FK])
- Building (Building_ID [PK], Building_Name)
- Schedule (Schedule_ID [PK], Student_ID [FK], Class_ID [FK])
- Course (Course_ID [PK] , Course_Name)



Data

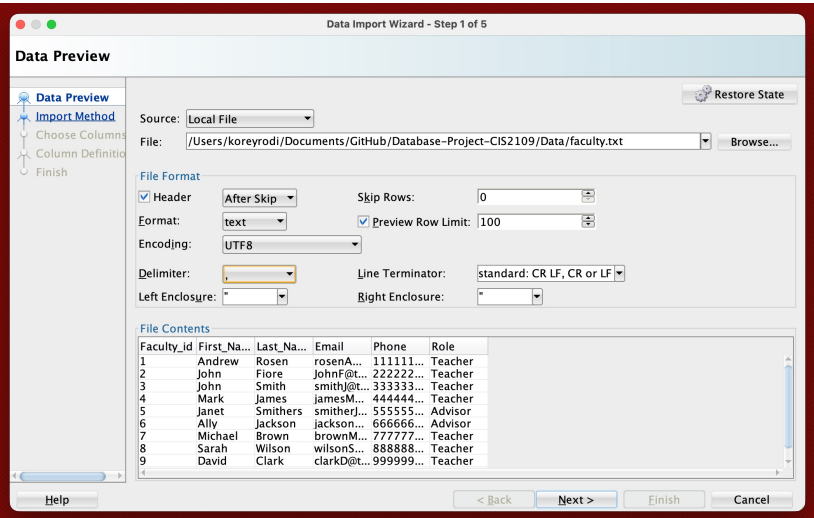
Adding Data

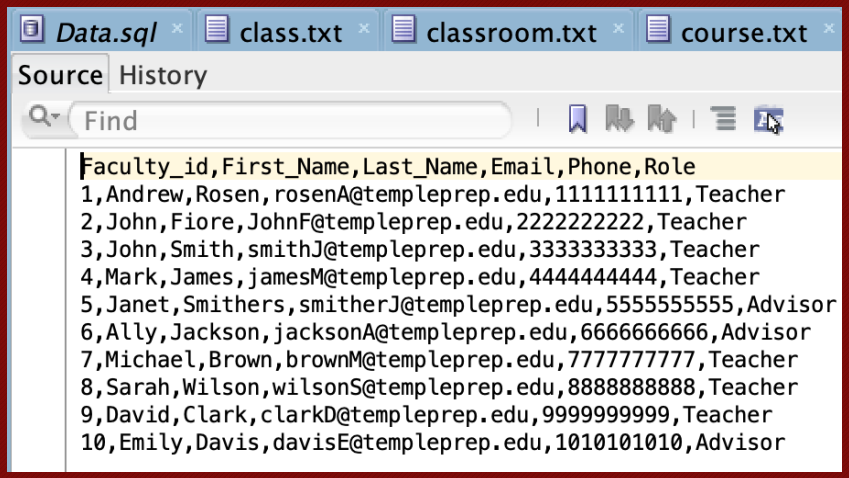
- Insert statements are best used for small datasets or small additions
- Data added this way will typically require you to understand the table structure (column names, data types, and whether values can be NULL)
- The order of values matter*
- Constraint rules such as primary keys (Unique, not null) and foreign keys must match



Adding Data cont.

- For larger tables or datasets bulk insert can be utilized
- .txt or .csv files are created with the header being the column names, and subsequent rows containing the records
- Bulk insert is built into SQL developer via the data import wizard which allows you to add large amounts of data set the delimiter typically a comma
- When importing data via this method all types of constraints are still enforced
- Bulk operations are much faster and scalable then insert statements

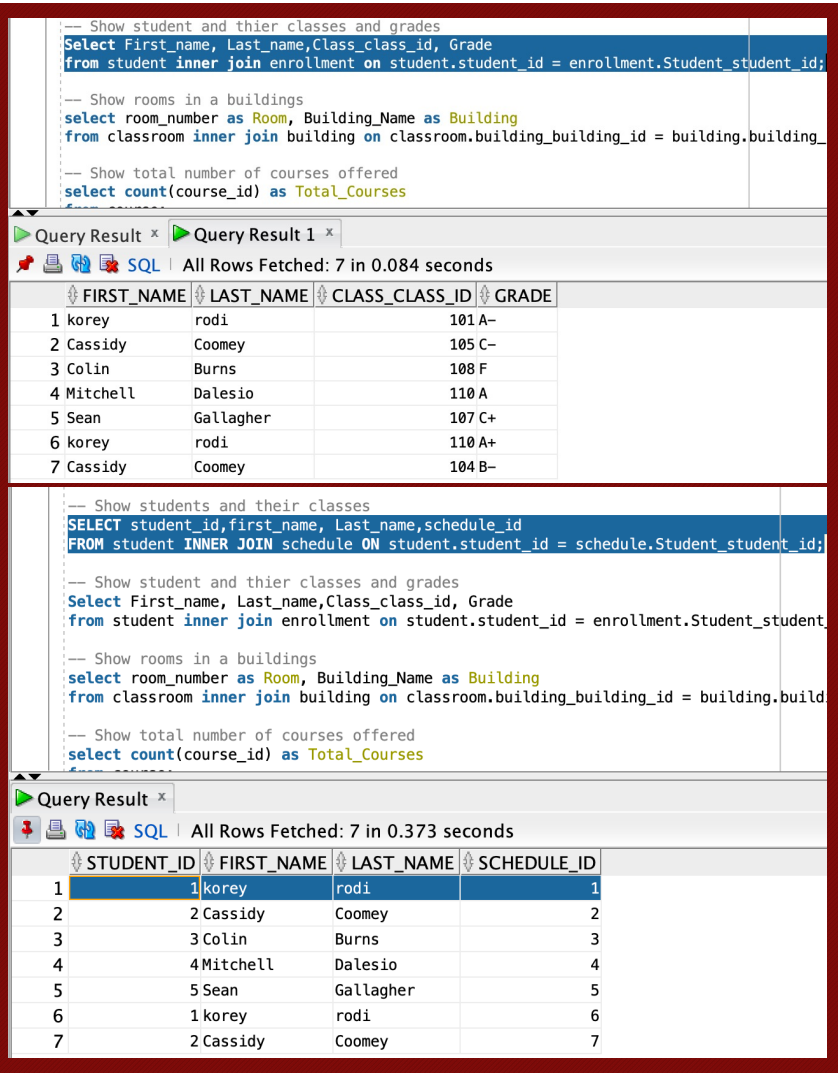




Showcasing Data

- Once data has been entered select statements can be used to show the data in valuable ways...

- Using joins (Left, Right, Inner, Natural, Full)
- Select * statements
- Aggregate functions can also be used to show things such as a count of the total number of classes
- Where clause
- Order by
- Group by (Used with aggregate functions)



```
-- Show student and thier classes and grades
Select First_name, Last_name,Class_class_id, Grade
from student inner join enrollment on student.student_id = enrollment.Student_student_id;

-- Show rooms in a buildings
select room_number as Room, Building_Name as Building
from classroom inner join building on classroom.building_building_id = building.building_

-- Show total number of courses offered
select count(course_id) as Total_Courses
```

Query Result | Query Result 1

SQL | All Rows Fetched: 7 in 0.084 seconds

	FIRST_NAME	LAST_NAME	CLASS_CLASS_ID	GRADE
1	korey	rodi	101	A-
2	Cassidy	Coomey	105	C-
3	Colin	Burns	108	F
4	Mitchell	Dalesio	110	A
5	Sean	Gallagher	107	C+
6	korey	rodi	110	A+
7	Cassidy	Coomey	104	B-

```
-- Show students and their classes
SELECT student_id,first_name, Last_name,schedule_id
FROM student INNER JOIN schedule ON student.student_id = schedule.Student_student_id;

-- Show student and thier classes and grades
Select First_name, Last_name,Class_class_id, Grade
from student inner join enrollment on student.student_id = enrollment.Student_student_

-- Show rooms in a buildings
select room_number as Room, Building_Name as Building
from classroom inner join building on classroom.building_building_id = building.build

-- Show total number of courses offered
select count(course_id) as Total_Courses
```

Query Result

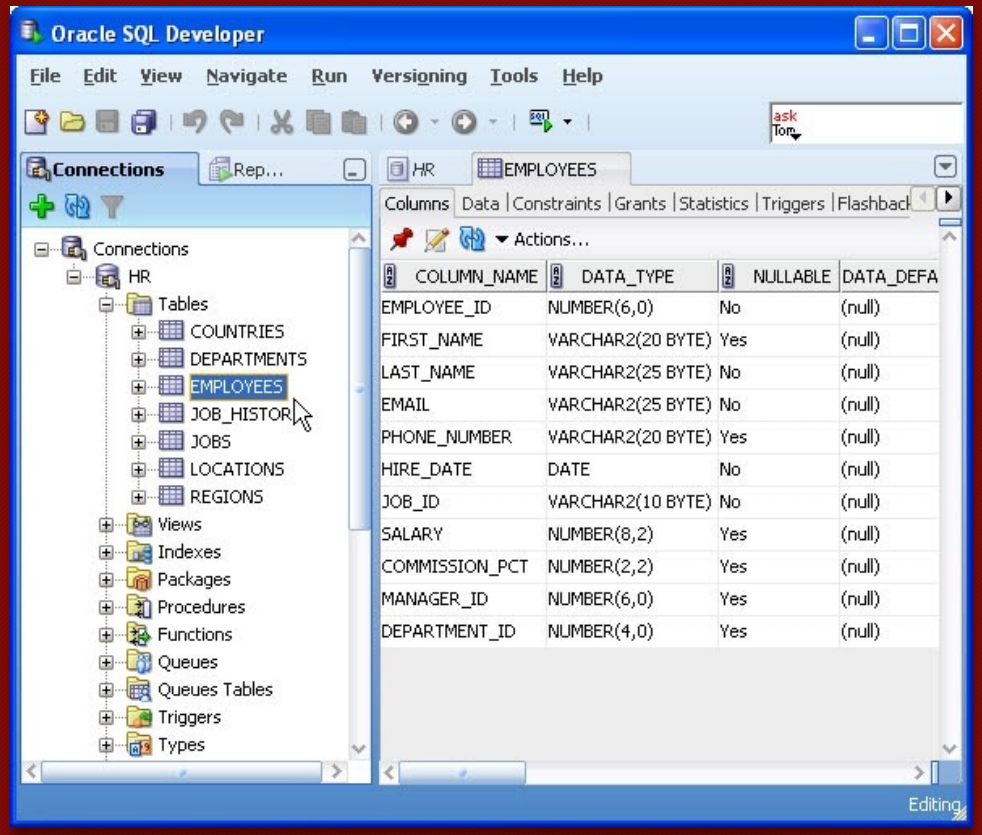
SQL | All Rows Fetched: 7 in 0.373 seconds

	STUDENT_ID	FIRST_NAME	LAST_NAME	SCHEDULE_ID
1	1	korey	rodi	1
2	2	Cassidy	Coomey	2
3	3	Colin	Burns	3
4	4	Mitchell	Dalesio	4
5	5	Sean	Gallagher	5
6	1	korey	rodi	6
7	2	Cassidy	Coomey	7

Challenges

Challenges

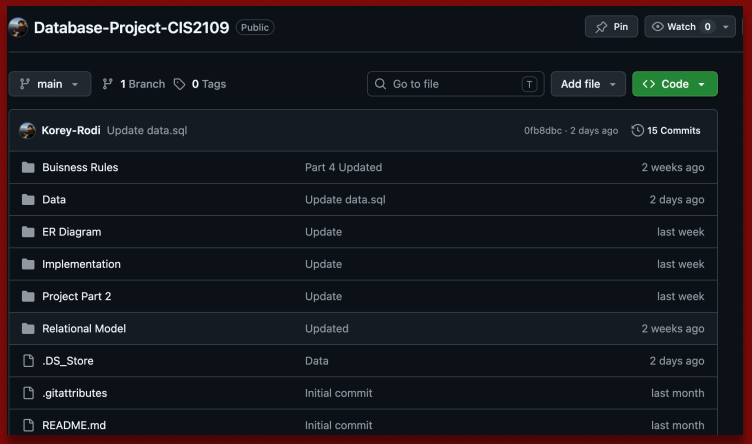
- Oracle SQL developer
- Older software steep learning curve to navigate and use
- Oracle SQL error messages
- DDL scripts
- Corrections being made to ensure database objects are created correctly
- Importing data
- Failed bulk inserts (column mismatch)



Successes

Successes

- Documentation
- Project deliverables were easily managed using git
- SQL
- Better understanding of all aspects of SQL such as software, Syntax, DDL and DML
- Business
- Creating a company idea
- Building out a database completely (ER, Implementation, Data, Physical Design, Users)




THANK
YOU